Exploring the WLasso Approach to Regularization

Mariana Garcia Mejia

November 10, 2026

1 Reproducing WLasso on simulated data

The paper A variable selection approach for highly correlated predictors in high-dimensional genomic data proposes a new approach to perform regularization in data sets with highly correlated variables. When variables in a dataset are highly correlated, the **Irrepresentable Condition (IC)**, fails to hold. This means that non-active variables can be represented as a combination of active variables, and variable selection techniques like Lasso are unable to tell apart the real signals. For instance, if X1 and X2 can explain the outcome Y, and X3 can be represented in terms of X1 and X2, then X3 will line up with Y as well. The WLasso technique transforms the data, the design matrix in particular, to ensure that the Irrepresentable Condition holds.

The following bullet points highlight characteristics to keep in mind:

- High-dimensionality: p » n
- Observations are independent (Fish 1 info. tells me nothing about Fish 2, likely doesn't hold in the fish dataset).
- X is highly correlated and the IC doesn't hold.

1.1 Simulating the data

Section 2.1 of the paper describes a simulation approach that we attempt to reproduce in the first part of this write up. They simulate a dataset with $n = 50$, and $p = 500$ where 10 variables are active and the rest are non-active. For simplicity purposes, we take $p = 100$. Additionally, they use a pre-set correlation matrix with correlation between active variables (SS) $\alpha_1 = 0.3$, correlation between active and non-active variables (S S^c) $\alpha_2 = 0.5$, and correlation between non-active variables (S^c S^c) $\alpha_3 = 0.7$. As shown below:

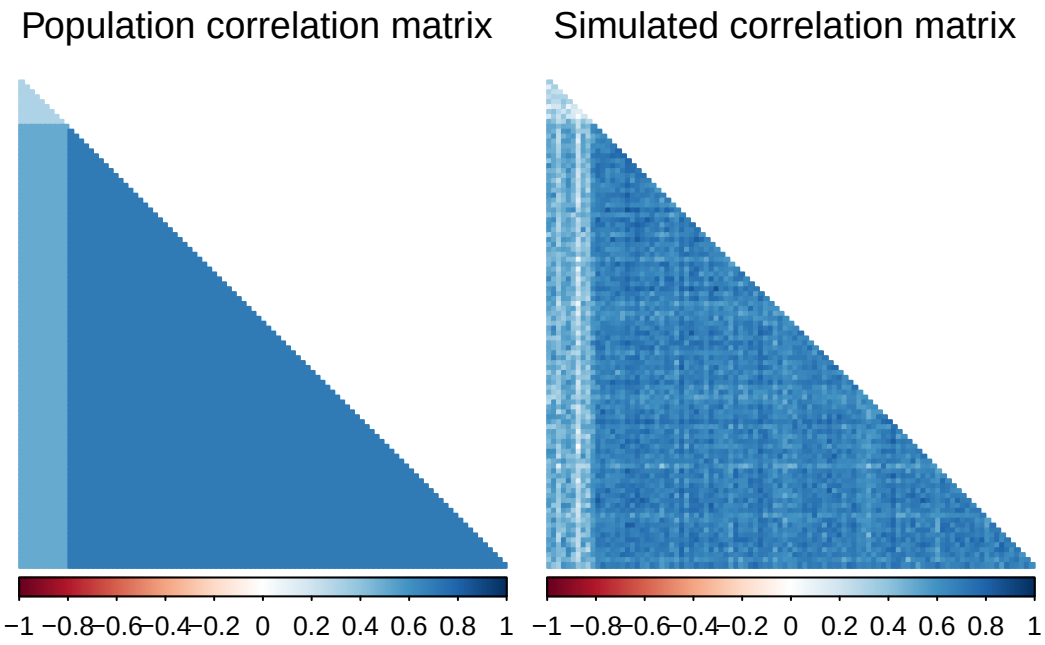
$$R = \begin{pmatrix} R_{SS} & R_{SS^c} \\ R_{S^cS} & R_{S^cS^c} \end{pmatrix} = \begin{pmatrix} 1 & 0.3 & \cdots & 0.3 & 0.5 & \cdots & 0.5 \\ 0.3 & 1 & \cdots & 0.3 & 0.5 & \cdots & 0.5 \\ \vdots & \vdots & \ddots & \vdots & \vdots & & \vdots \\ 0.3 & 0.3 & \cdots & 1 & 0.5 & \cdots & 0.5 \\ 0.5 & 0.5 & \cdots & 0.5 & 1 & 0.7 & \cdots \\ \vdots & \vdots & & \vdots & 0.7 & \ddots & \vdots \\ 0.5 & 0.5 & \cdots & 0.5 & \cdots & \cdots & 1 \end{pmatrix}$$

The paper describes they simulate independent Gaussian vectors with covariance matrix R. Hence we simulate $X_i \sim N(0, R)$ to form the design matrix X. Note that β contains 10 active variables with value 2, the rest are non-active so the coefficients are zero. The active variables are in the 10 first entries of the β matrix to match with the correlation matrix. We use the design matrix and the coefficients from above to predict:

$$y = X\beta + \epsilon$$

with

$$X_i \overset{\text{iid}}{\sim} N(0, R), \qquad \beta_j = \begin{cases} 2, & j \in S, \\ 0, & j \in S^c. \end{cases}$$



1.2 Checking the Irrepresentable Condition

First, we check that the Irrepresentable Condition does not hold. As previously mentioned, the Irrepresentable Condition (IC) is an important condition for the Lasso to consistently identify the true set of nonzero coefficients. It requires that the inactive predictors cannot be too strongly represented by the active predictors.

Let $S = \{j : \beta_j \neq 0\}$ denote the set of active predictors and S^c the set of inactive predictors. The IC is defined as:

$$\left| X_{S^c}^T X_S (X_S^T X_S)^{-1} \operatorname{sign}(\beta_S) \right|_j \leq 1 - \gamma, \qquad \forall j \in S^c,$$

where $\gamma \in (0, 1]$.

Here:

- X_S contains the columns of X corresponding to active predictors.
- X_{S^c} contains the columns corresponding to inactive predictors.
- $\operatorname{sign}(\beta_S)$ is the vector of signs of the nonzero coefficients.
- γ represents a positive margin ensuring that the condition is strictly satisfied.

In our simulation, we have 10 active and 90 inactive predictors. Therefore, the expression produces a vector of 90 values, one for each inactive predictor.

The maximum IC value obtained from our simulated data was 1.6816, exceeding the required threshold of 1. Therefore, the Irrepresentable Condition is violated. Furthermore, 86 out of 90 inactive predictors (95.56%) have IC values greater than or equal to 1. This suggests that the correlation structure makes it difficult for the Lasso to distinguish relevant from irrelevant predictors, potentially compromising its variable selection performance.

1.3 Fitting the regular Lasso

Next, we fit the regular Lasso to have a baseline comparison for WLasso. The regular Lasso minimizes the MSE with an L1 penalty. This enforces sparsity more compared to for instance Ridge Regularization. WLasso benefits from this.

$$\widehat{\beta}^{\text{Lasso}} = \arg\min_{\beta} \left\{ \|y - X\beta\|_2^2 + \lambda \|\beta\|_1 \right\},$$

where

$$\|\beta\|_1 = \sum_{j=1}^{p} |\beta_j|.$$

Table 1: Classical Lasso variable selection results on the simulated dataset.

Metric	Value
True Positives (TP)	10
False Positives (FP)	23
False Negatives (FN)	0
Total Selected	33

The Lasso identified 33 total variables: all 10 active variables correctly, and 23 additional non-active variables. These results illustrate the difficulty of variable selection in the presence of highly correlated predictors. Although Lasso successfully recovered all relevant variables, it failed to exclude many irrelevant ones.

1.4 Introducing WLasso

The objective of the whitening transformation is to remove correlations between predictors, addressing one of the main difficulties faced by classical Lasso when the Irrepresentable Condition (IC) is violated.

Let $R \in \mathbb{R}^{p \times p}$ denote the population correlation matrix of the predictors.

Since R is symmetric and positive definite, we can apply eigendecomposition:

$$R = UDU^T,$$

where:

- U is an orthogonal matrix whose columns are the eigenvectors of R, satisfying $U^TU = UU^T = I_p$.
- $D = \text{diag}(d_1, \dots, d_p)$ is a diagonal matrix containing the positive eigenvalues of R.

We define the whitened design matrix as:

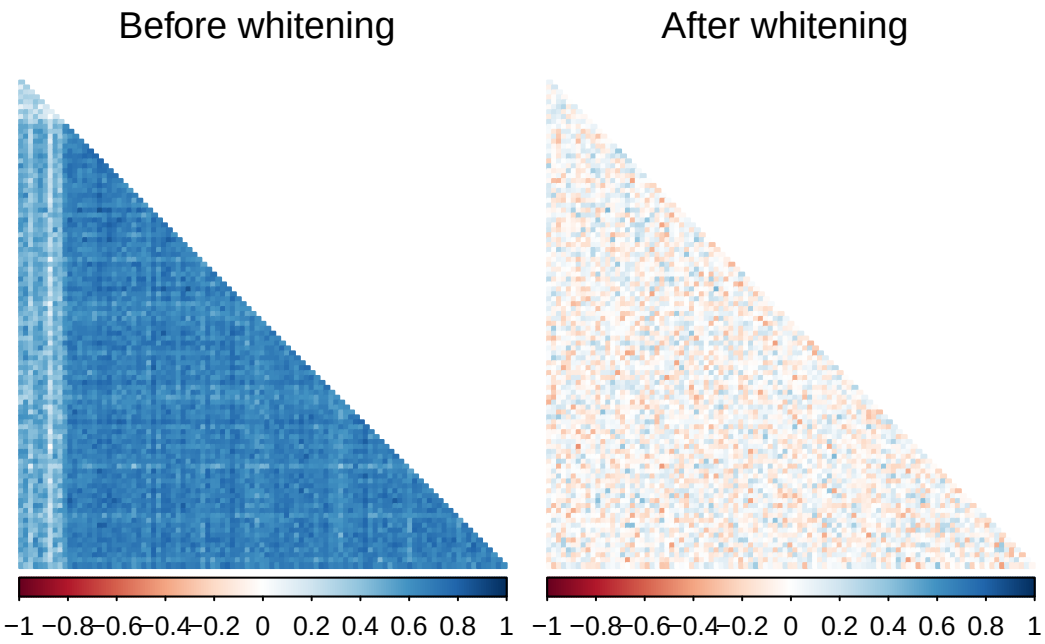
$$\widetilde{X} = XR^{-1/2}, \qquad R^{-1/2} = UD^{-1/2}U^T$$

To understand why this transformation removes correlations, consider the covariance matrix of a row of X. Assuming $\text{Cov}(X_i) = R$, we obtain:

$$\begin{aligned} \text{Cov}(\widetilde{X}_i) &= R^{-1/2} \,\text{Cov}(X_i) R^{-1/2} \\ &= R^{-1/2} R R^{-1/2} \\ &= I_p. \end{aligned}$$

Therefore, the transformed predictors have an identity population covariance matrix. This means that the columns of $\widetilde{X}$ are uncorrelated in the population.

Importantly, since our simulated dataset has only $n = 50$ observations, the empirical correlation matrix $\text{cor}(\widetilde{X})$ will not be exactly equal to I_p. Form our simulated data, the design matrix results in the correlation matrix below, which is much closer to the identity matrix (uncorrelated variables).



Recall our original regression model:

$$y = X\beta + \epsilon.$$

We can insert the identity matrix $R^{-1/2}R^{1/2} = I_p$:

$$\begin{aligned} y &= X\beta + \epsilon \\ &= XR^{-1/2}R^{1/2}\beta + \epsilon \\ &= \widetilde{X}\tilde{\beta} + \epsilon \end{aligned}$$

We therefore define:

$$\widetilde{X} = XR^{-1/2} \qquad \text{and} \qquad \tilde{\beta} = R^{1/2}\beta$$

The error term ϵ remains unchanged because we are only transforming the predictors and coefficients, not the response variable.

Important: Although the original coefficient vector β is sparse, the transformed coefficient vector $\tilde{\beta}$ is generally not sparse. This happens because multiplying by $R^{1/2}$ mixes information across coefficients.

Consequently, applying ordinary Lasso directly to $\widetilde{X}$ would not necessarily recover the original sparse coefficient vector. WLasso addresses this by modifying the penalty, which we will derive in the next step.

1.5 Deriving the Generalized Lasso Penalty

The classical Lasso minimizes the following objective function:

$$\hat{\beta} = \arg\min_{\beta} \left\{ \|y - X\beta\|_2^2 + \lambda \|\beta\|_1 \right\}.$$

Recall the whitening transformations:

$$\widetilde{X} = XR^{-1/2}, \qquad \tilde{\beta} = R^{1/2}\beta.$$

Rearranging the second transformation gives:

$$\beta = R^{-1/2}\tilde{\beta}.$$

Substituting into the original Lasso objective:

$$\|y - X\beta\|_2^2 + \lambda\|\beta\|_1 = \|y - \widetilde{X}\tilde{\beta}\|_2^2 + \lambda\|R^{-1/2}\tilde{\beta}\|_1.$$

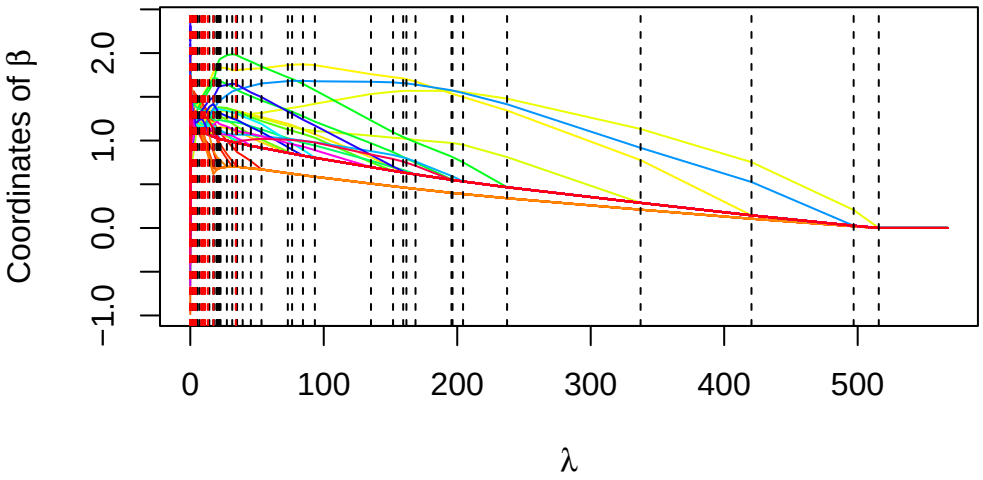
Therefore, the generalized Lasso objective becomes:

$$\hat{\tilde{\beta}} = \arg\min_{\tilde{\beta}} \left\{ \|y - \widetilde{X}\tilde{\beta}\|_2^2 + \lambda\|R^{-1/2}\tilde{\beta}\|_1 \right\}$$

Unlike ordinary Lasso applied to the whitened predictors, the generalized Lasso penalizes a linear transformation of the coefficients.

This is important because $\tilde{\beta}$ is generally not sparse, even when the original β is sparse. By penalizing $R^{-1/2}\tilde{\beta} = \beta$, the generalized Lasso encourages sparsity in the original coefficient vector, not the transformed one. We use the package `genlasso` instead of `glmnet` because we penalize a linear transformation of coefficients.

First, we fit the Lasso without any parameter tunning. Below, each colored line is a coefficient as lambda increases. Some go to zero quicker than others (for smaller lambdas). So, at this moment, we have the different estimated $\tilde{\beta}$ for each value of λ.



Notes:

- At this point, we do not select an optimal value of λ, yet. In the paper we apply a double thresholding procedure.
- The double thresholding procedure is used because we must not only select non-sparse coefficients in the transformed space, but also in the real space. Once we find beta coefficients in the transformed space $\tilde{\beta}$s using Lasso, we must take them back to the original space using $\beta = R^{-1/2}\tilde{\beta}$.

1.6 Double thresholding

The double-thresholding procedure consists of the following steps for each candidate penalty parameter λ:

1. **Extract the estimated whitened coefficients.** Obtain $\hat{\tilde{\beta}}(\lambda)$ from the generalized Lasso solution path.
2. **First thresholding (K).** Retain the K largest coefficients in absolute value, preserving their estimated values. The remaining coefficients are replaced by the K-th largest absolute coefficient, while preserving their original signs.

For example, suppose:

$$\tilde{\hat{\beta}} = \begin{pmatrix} 2.0 & -1.5 & 0.8 & -0.3 \end{pmatrix}^T.$$

If $K = 2$, the thresholded coefficients become:

$$\tilde{\hat{\beta}}^{(2)} = \begin{pmatrix} 2.0 & -1.5 & 1.5 & -1.5 \end{pmatrix}^T.$$

3. **Recover the original coefficients.** Map the thresholded estimates back to the original coordinate system:

$$\hat{\beta}^0(\lambda) = R^{-1/2}\tilde{\hat{\beta}}^{(K)}(\lambda).$$

4. **Second thresholding (M).** Retain the M largest recovered coefficients in absolute value and set all remaining coefficients to zero. This produces the final sparse estimate $\hat{\beta}^{(M)}(\lambda)$.

1.7 Selecting the Tuning Parameters

For each λ, the parameters K and M are selected using the paper's MSE-ratio criterion, which measures whether retaining an additional coefficient produces a sufficiently large reduction in error.

The threshold c controls this decision. For example, $c = 0.95$ means that an additional coefficient must reduce the error by more than approximately 5% to justify increasing the number of retained coefficients.

Once K and M have been determined for each candidate λ, we calculate:

$$\mathrm{MSEM}(\lambda) = \left\| y - X\hat{\beta}^{(M)}(\lambda) \right\|_2^2.$$

Finally, we select:

$$\boxed{\lambda_{\mathrm{opt}} = \arg\min_{\lambda} \mathrm{MSEM}(\lambda)}$$

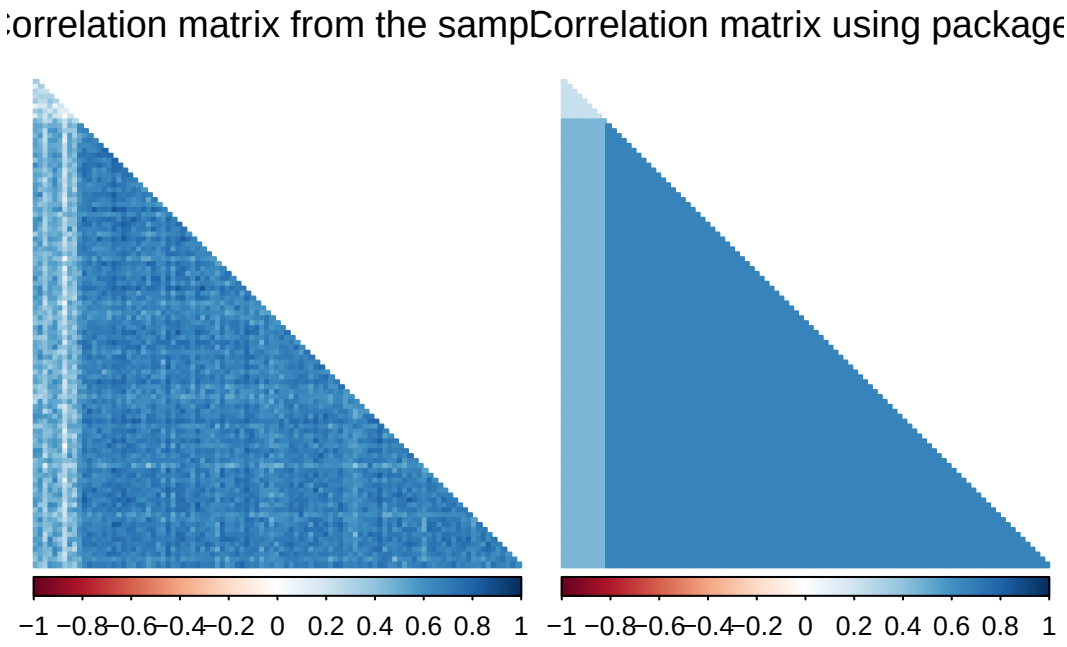
Thus, the procedure selects K and M for each candidate λ, evaluates the resulting model, and then chooses the optimal penalty parameter.

This algorithm is more complex to fit manually, so we use the package.

1.8 Using the WLasso package

The package is called WLasso. It contains functions: Sigma Estimation, top, top_thresh, Whitening_Lasso.

We start by using the `Sigma_Estimation(X)` function, to observe how it differs from the normal `Cor(X)` in estimating the correlation matrix.



The function in the package identifies $\alpha_1 = 0.2249$, compared to 0.3 in the population, $\alpha_2 = 0.45$ compared to 0.5 in the population, and $\alpha_3 = 0.66$ compared to 0.7 in the population. Note that the package returns this using `$alpha` and in order of: inactive-inactive, inactive-active, and active-active. The function identifies groups 1–9 and 10–100, closely approximating the true partition of 1–10 and 11–100. Note that the package uses hierarchical clustering to partition predictors into two groups, this is not variable selection yet.

Next, we use `Whitening_Lasso()` to find the coefficient estimates. Note that all steps happen inside the function, we provide it the `X` design matrix, the `Y` response vector, the estimated correlation matrix `R_hat`, and we select a `gamma` value. Authors report values between 0.9 and 0.99 perform well, this is the threshold value at which an increase in MSE is worth adding an additional predictor or not.

$\lambda = 3$ minimizes MSE, at $MSE = 1.04$. The method identified all 10 active coefficients correctly, and 4 additional ones. In general, this method results better than regular Lasso in terms of variable selection (regular Lasso identified 23 variables; 13 false positives).

Table 2: WLasso variable selection results on the simulated dataset.

Metric	Value
True Positives (TP)	10
False Positives (FP)	4
False Negatives (FN)	0
Total Selected	14

1.9 Evaluating the package MSE Calculation

```
  MSE_Package MSE_Manual Difference
1    1.036064  0.7937831  0.2422811
```

Note that at the optimal lambda, and using the same estimated coefficients, the MSE estimated by the package and the manual MSE are not the same.

